

Grupo 3

N	Concepto	Puntaje Máximo	Asignado	Asignado	Calificación
1	Introducción	3	3	94	6,6
2	Contexto del Problema y Planteamiento Inicial	3	3		
3	Metodología de Desarrollo	5	5		
4	Construcción de la Base de Datos	10	10		
5	Modelamiento del Problema	8	8		
6	Desarrollo del Algoritmo	15	13		
7	Incorporación de Restricciones Reales	7	7		
8	Modelación del Embalaje y Volumen	3	3		
9	Resultados Obtenidos	8	8		
10	Validación del Modelo	10	10		
11	Análisis Crítico del Proceso	10	10		
12	Lecciones Aprendidas	5	3		
13	Limitaciones del Trabajo	3	3		
14	Trabajo Futuro	5	5		
15	Conclusiones	5	3		

Informe Académico: Programación de un CVRPTW en una aplicación integrada para una empresa



Parraud, Joaquín; Ramírez, Vicente; Ramírez, Sebastián

PhD. Victor Vera Valdés

IIC511AD Capstone Analytics

13 de Abril de 2026

Resumen Ejecutivo

El presente informe detalla el diseño, desarrollo e implementación de un Sistema de Gestión de Entregas (DMS) basado en el Problema de Ruteo de Vehículos con Ventanas de Tiempo y Capacidad (CVRPTW). El proyecto nace como respuesta a los desafíos logísticos de una empresa importadora en expansión, la cual enfrentaba sobrecostos operacionales, entregas fallidas y un desgaste general de su flota debido a la planificación manual y empírica de sus rutas diarias.

Para abordar este desafío matemático de complejidad NP-Hard, el equipo adoptó un enfoque metodológico iterativo y una arquitectura orientada a Software as a Service (SaaS), separando estrictamente la interfaz de usuario de la lógica algorítmica. El pipeline de procesamiento integró la limpieza, normalización y geocodificación de un conjunto de datos real compuesto por 4.180 pedidos en la Región Metropolitana.

El núcleo de optimización se construyó mediante un enfoque híbrido: una heurística constructiva basada en Solomon II para garantizar la generación rápida de rutas iniciales factibles, seguida de la metaheurística Tabu Search (Búsqueda Tabú) encargada de la minimización del costo. El modelo matemático se diseñó para respetar restricciones operativas del mundo real, incluyendo capacidades vehiculares de peso y volumen (ej. 1.850 kg y 12 m³), tiempos de servicio escalados logarítmicamente según el tamaño del pedido, y un estricto límite legal de 5 horas (300 minutos) de conducción continua.

Los resultados experimentales demostraron la alta eficiencia y aplicabilidad de la solución. La configuración base del algoritmo logró procesar 35 días de operación logística, alcanzando una reducción de costos de traslado del 4,2% en comparación con la heurística base, requiriendo apenas 20 segundos de presupuesto computacional para la búsqueda local. Además, las pruebas de estrés (escalando hasta 800 pedidos y simulando saturación de flota) validaron la robustez del sistema, el cual logra fallar de manera controlada y priorizar la factibilidad operativa frente a demandas inviables.

En conclusión, el proyecto cumplió a cabalidad sus objetivos académicos y técnicos. Se logró entregar una plataforma funcional, escalable y parametrizable que abstrae la complejidad matemática subyacente, reduciendo el esfuerzo de planificación del cliente y sentando las bases tecnológicas para futuras integraciones con tráfico dinámico y modelos predictivos de demanda.

Tabla de Contenido

1	Introducción	1
2	Contexto del Problema y Planteamiento Inicial.....	2
	Supuestos Iniciales.....	2
	Nivel de Complejidad Percibido	2
	Decisiones Iniciales de Modelamiento	3
3	Metodología de Desarrollo	4
4	Construcción de la Base de Datos.....	6
5	Modelamiento del Problema (CVRPTW).....	8
5.1	Formulación	8
5.2	Conjuntos, parámetros e índices	8
5.3	Variables	9
5.4	Restricciones y Función Objetivo	9
5.5	Simplificaciones.....	10
5.6	Reflexión del modelo.....	12
6	Desarrollo del Algoritmo	13
7	Incorporación de Restricciones Reales	19
8	Modelación del Embalaje, Volumen y Peso	19
9	Resultados Obtenidos.....	20
10	Validación del Modelo.....	22
10.1	Pruebas realizadas, sus propósitos y sus resultados.....	22
10.2	Coherencia de resultados	23
10.3	Comparación con expectativas	24
11	Análisis Crítico del Proceso de Desarrollo del Proyecto	25
11.1	Aciertos Metodológicos y Técnicos.....	25
11.2	Desaciertos y Oportunidades de Mejora	25
11.3	Decisiones Clave que Definieron el Proyecto	25
12	Lecciones Aprendidas	27
13	Limitaciones del Trabajo	28

14	Trabajo Futuro.	29
15	Conclusiones	30
16	Referencias.....	31

1 Introducción

La optimización de costos logísticos a través del Problema de Ruteo de Vehículos (VRP) representa un desafío clásico en la investigación de operaciones. Clasificado dentro de la complejidad NP-Hard, el VRP exige un esfuerzo computacional que crece de forma exponencial respecto al número de nodos, haciendo inviables los métodos de resolución exacta para operaciones logísticas a gran escala. Para mitigar esta barrera, la industria y la literatura académica recurren a la hibridación de heurísticas constructivas y metaheurísticas de búsqueda local.

El objetivo de este proyecto es encapsular esta complejidad algorítmica dentro de un Delivery Management System (DMS) funcional y eficiente. La plataforma desarrollada busca no solo resolver el problema de enrutamiento con bajos tiempos de latencia, sino también democratizar el acceso a estas herramientas matemáticas mediante un dashboard interactivo. De este modo, se proporciona una solución integral que reduce el esfuerzo operativo del cliente en la planificación y visualización de sus rutinas logísticas diarias.

Objetivo General: Aplicar e integrar de manera práctica los conocimientos adquiridos durante la carrera en las áreas de investigación de operaciones, ciencias de datos e ingeniería de software, mediante el diseño, implementación y validación de una solución tecnológica real para un problema logístico complejo de ruteo de vehículos (VRPTW).

Objetivos Específicos:

Modelamiento Matemático: Traducir las restricciones operativas reales de una empresa (ventanas de tiempo, capacidades, jornada laboral) en un modelo matemático estructurado que permita su resolución computacional.

Implementación Algorítmica: Programar, calibrar y evaluar el desempeño de métodos de resolución aproximados (heurísticas constructivas y metaheurísticas de búsqueda local) frente a la intratabilidad de los métodos exactos en instancias NP-Hard.

Arquitectura de Sistemas: Diseñar un pipeline de datos robusto que abarque desde la limpieza y geocodificación de la información (ETL), hasta la ejecución de la lógica de optimización en un entorno backend desacoplado de la interfaz de usuario.

Presentacion de resultados: Crear e implementar un dashboard que muestre al cliente de forma efectiva los resultados obtenidos por el modelo para su problema de optimización VRP.

2 Contexto del Problema y Planteamiento Inicial

El caso de estudio presenta el escenario de una empresa importadora en expansión que enfrenta una crisis operativa derivada del manejo interno (in-house) de su logística de distribución. Si bien la selección de productos ha impulsado las ventas, la incapacidad de la empresa para escalar su modelo de despacho ha generado sobrecostos, alta rotación de personal (debido a jornadas laborales extensas y desequilibradas), desgaste de la flota actual y un deterioro general en la calidad del servicio al cliente (entregas fallidas, pedidos incompletos y rechazos).

Supuestos Iniciales

Al inicio del análisis, y previo a la estructuración matemática del modelo, se establecieron los siguientes supuestos simplificadores:

Política de Ventanas de Tiempo (Flexibilidad Temporal): Inicialmente, se asumió que no existían ventanas de tiempo estrictas para las entregas (Time Windows). Se supuso que una notificación previa al cliente sobre el día de despacho era suficiente para garantizar la recepción, asimilando el problema a un ruteo clásico sin restricciones horarias.

Capacidad y Homogeneidad de Flota: Se partió del supuesto de que el parque automotriz debía renovarse por completo, asumiendo (en una primera iteración) una flota homogénea disponible, sin un estudio previo sobre el mix óptimo de capacidades.

Aislamiento de la Congestión Urbana: El impacto de la ubicación subóptima del actual Centro de Distribución y las externalidades del tráfico se consideraron, inicialmente, como factores externos a mitigar de forma heurística, sin integrarlos dinámicamente en el costo de los arcos del grafo de la red vial.

Nivel de Complejidad Percibido

El desafío logístico descrito se identificó rápidamente como un Problema de Ruteo de Vehículos (VRP). A medida que se incorporaron las necesidades reales del negocio —como las restricciones de jornada laboral máxima y la posterior integración de franjas horarias de recepción para evitar entregas fallidas— el modelo escaló a la variante VRPTW (Vehicle Routing Problem with Time Windows).

Esta familia de problemas de optimización combinatoria está clasificada como NP-Hard. La complejidad inherente radica en que el esfuerzo computacional para encontrar el óptimo global crece exponencialmente con el tamaño de la instancia (número de pedidos a despachar). Para operaciones a escala comercial como la descrita, los métodos de resolución exacta resultan computacionalmente inviables, planteando un desafío técnico significativo en la búsqueda de soluciones operativamente factibles en tiempos razonables de ejecución.

Decisiones Iniciales de Modelamiento

Ante la complejidad NP-Hard detectada, las decisiones arquitectónicas tempranas buscaron simplificar el núcleo algorítmico mediante la descomposición del problema:

Enfoque Heurístico: Se descartó el uso de modelos de programación lineal entera mixta (MILP) exactos. La decisión fue adoptar heurísticas constructivas para generar soluciones iniciales rápidas, y metaheurísticas de búsqueda local para mejorar la calidad del ruteo.

Relajación de Ventanas de Entrega: Como decisión de simplificación inicial, se optó por ignorar temporalmente las ventanas horarias de los clientes, resolviendo el problema como un Capacitated VRP (CVRP) estándar.

Segregación de Complejidad (Pre/Post-Procesamiento): Se decidió que cualquier requerimiento logístico avanzado (como la consolidación de pedidos atrasados, la reasignación de pedidos incompletos o el balanceo de carga laboral) se gestionaría fuera del optimizador central. Estos elementos se tratarían mediante reglas de negocio previas a la optimización (pre-procesamiento de la data) o ajustes posteriores a la creación de las rutas (post-procesamiento), manteniendo el motor algorítmico lo más ligero posible.

3 Metodología de Desarrollo

El desarrollo del proyecto se abordó mediante una metodología iterativa orientada a la construcción progresiva de una solución funcional, escalable y coherente con las restricciones reales del problema logístico estudiado. Desde un inicio, se definió como prioridad construir un sistema capaz de integrar preparación de datos, modelamiento matemático, heurísticas de resolución y visualización de resultados dentro de un mismo flujo operativo. En consecuencia, el desarrollo del código no se planteó únicamente como la implementación de un algoritmo de optimización, sino como el diseño de un pipeline completo de apoyo a la toma de decisiones, donde cada etapa dependiera de la calidad y consistencia de la etapa anterior.

La primera fase del trabajo estuvo centrada en la generación y preparación de los datos requeridos por el modelo. Dado que el problema de ruteo depende críticamente de la calidad de la información de entrada, se diseñó un proceso de validación, limpieza y normalización que permitiera transformar archivos operacionales heterogéneos en una base utilizable por el optimizador. Esta etapa incluyó la revisión de formatos, columnas obligatorias, codificación de caracteres, estandarización de direcciones y comunas, y vinculación entre órdenes de venta y su respectivo detalle. A partir de ello, fue posible construir una representación estructurada de la demanda diaria, expresada en términos de peso, volumen, ubicación geográfica y fecha comprometida de despacho.

Una vez consolidada la información base, se desarrolló el modelamiento inicial del problema como una variante del problema de ruteo de vehículos con restricciones de capacidad y tiempo. Esta formulación permitió definir con claridad las entidades relevantes del sistema, sus parámetros operacionales y las restricciones que debían respetarse durante la planificación. En esta fase se procuró capturar la estructura esencial del problema sin sobrecargar el modelo con elementos cuya incorporación temprana pudiera comprometer la trazabilidad computacional. El objetivo fue establecer una base formal suficientemente realista para representar la operación, pero lo bastante simple para permitir el desarrollo eficiente de métodos de solución escalables.

Sobre esta base se avanzó al desarrollo del algoritmo de resolución. Inicialmente se exploró la posibilidad de utilizar un enfoque exacto, observándose un comportamiento adecuado en instancias de menor tamaño. Sin embargo, al aumentar la escala del problema, se hizo evidente la necesidad de adoptar estrategias heurísticas y metaheurísticas que permitieran mantener tiempos de respuesta compatibles con un entorno operativo real. En este contexto, se seleccionó una heurística constructiva tipo Solomon II para generar una solución inicial factible, la cual posteriormente fue mejorada mediante procedimientos de búsqueda local y Tabu Search. Esta arquitectura algorítmica respondió a una lógica práctica: obtener primero una solución rápida y estructuralmente consistente, para luego refinarla mediante mecanismos de exploración del vecindario que mejoraran su desempeño en términos de costo y calidad de rutas.

La validación del desarrollo se realizó de manera continua, tanto a nivel de consistencia lógica como de desempeño operacional. Cada módulo fue evaluado en función de su capacidad para entregar salidas coherentes con los supuestos del sistema, verificando que la limpieza produjera

datos procesables, que la geocodificación generara coordenadas razonables, que las matrices de distancia y tiempo reflejaran adecuadamente la red vial y que las rutas obtenidas fueran factibles respecto de las restricciones del modelo. Esta validación no se restringió al algoritmo en sí, sino que consideró el comportamiento integral del sistema, incluyendo la generación de mapas, archivos exportables y métricas de desempeño que permitieran interpretar los resultados desde una perspectiva logística y económica.

Finalmente, el proyecto incorporó una fase permanente de ajustes y refinamiento, motivada por los resultados observados en pruebas sucesivas. Estos ajustes incluyeron la calibración de parámetros del algoritmo, la incorporación de mecanismos de postoptimización, la reasignación de rutas a camiones físicos y la introducción de aproximaciones para representar mejor el tráfico y la operación diaria. En lugar de buscar desde el inicio una formulación exhaustiva de la realidad, se optó por un desarrollo incremental, donde las mejoras se integraron en función de su aporte práctico al sistema y de su costo computacional. De este modo, la metodología de desarrollo combinó rigurosidad analítica con criterio ingenieril, privilegiando una solución funcional, interpretable y progresivamente perfeccionable.

4 Construcción de la Base de Datos

La construcción de la base de datos constituyó una etapa clave en el desarrollo de la solución, no solo como soporte de almacenamiento, sino como parte integral del sistema de optimización. Se diseñó una base de datos relacional orientada a múltiples clientes, capaz de administrar de forma separada y segura la información asociada a cada usuario de la aplicación. Esta estructura permite integrar los archivos operacionales de entrada, almacenar parámetros y configuraciones específicas de cada cliente, registrar resultados de ejecuciones previas y conservar rutas procesadas en caché, todo ello dentro de una arquitectura persistente y reutilizable. Asimismo, al tratarse de una plataforma con acceso diferenciado por usuario, se incorporó un esquema de autenticación con credenciales cifradas, resguardando la información sensible y fortaleciendo la seguridad del sistema.

La base fue construida para integrar los dos archivos principales que alimentan la optimización: el archivo de ventas y direcciones, que contiene la información general de cada pedido, y el archivo de detalle de ventas, que contiene la composición física de la orden a nivel de producto. La estructura relacional permitió vincular ambos archivos mediante claves comunes, consolidando en una sola entidad lógica la información comercial y logística necesaria para el modelo.

Uno de los principales desafíos encontrados durante esta etapa estuvo asociado a la calidad e inconsistencia de los datos originales. Se detectaron clientes con nombres duplicados, clientes sin nombre registrado, direcciones inexistentes, direcciones ambiguas o incompletas y otros errores de registro que impedían su utilización directa en el modelo. Estas dificultades resultaban especialmente críticas en un problema de ruteo, donde la validez de cada punto de entrega depende de su correcta identificación y localización espacial. En consecuencia, fue necesario desarrollar una transformación de datos exhaustiva, orientada no solo a limpiar la base, sino a convertirla en una fuente confiable de información logística.

El tratamiento aplicado a los datos incluyó procesos de normalización, depuración y ajuste estructural. Se eliminaron duplicados, se corrigieron formatos inconsistentes y se estandarizaron nombres, comunas y direcciones para reducir ambigüedades. De manera particular, se realizaron ajustes sobre las direcciones con el objetivo de que pudieran ser correctamente interpretadas por el geolocalizador, mejorando así la tasa de éxito de la geocodificación y la calidad de las coordenadas obtenidas. Esta transformación dejó de ser una etapa externa o manual y pasó a formar parte del pipeline de procesamiento de la aplicación, de modo que actualmente se ejecuta de manera integrada y sistemática antes de cada optimización. Con ello, la base de datos no solo cumple una función de almacenamiento, sino también de estructuración y preparación de información para el modelo.

Adicionalmente, la base fue diseñada para almacenar parámetros y configuraciones propias de cada cliente, tales como cantidad de camiones disponibles, capacidades máximas, rendimiento de combustible, costos operacionales, ventanas horarias y opciones avanzadas del algoritmo. Esta persistencia permite que la aplicación mantenga continuidad entre ejecuciones y que distintos

usuarios puedan operar sobre la misma plataforma sin interferencia entre sus datos ni sus reglas de negocio. Junto con ello, se incorporó almacenamiento en caché de rutas y resultados procesados, lo que reduce reprocesamientos innecesarios y mejora la eficiencia general del sistema cuando existen corridas repetidas o escenarios similares.

En relación con la validación experimental, se construyeron múltiples instancias de prueba a partir de la segmentación de la base de datos y la generación de datos aleatorios controlados. Estas instancias fueron diseñadas para evaluar el comportamiento del sistema en escenarios de distinta complejidad y para someter el pipeline completo a condiciones de estrés crecientes. En total, se generaron aproximadamente cincuenta bases de datos de prueba, escalando progresivamente hasta escenarios de 800 clientes. La diversidad de estos conjuntos permitió analizar casos con clientes muy alejados entre sí, clientes ubicados a gran distancia del centro de distribución, zonas con alta concentración espacial de entregas y combinaciones intermedias de dispersión y agrupamiento. Esta variedad respondió al propósito de probar no solo la factibilidad del algoritmo, sino también la robustez de la base de datos y del proceso de transformación frente a configuraciones operacionales muy distintas entre sí.

5 Modelamiento del Problema (CVRPTW)

Esta sección presenta la formulación del problema de ruteo de vehículos como una representación simplificada del proceso diario de despacho. El modelo incorpora las principales restricciones operativas y de costo con el fin de minimizar el costo total de distribución como un CVRPTW.

5.1 Formulación

Se busca planificar los despachos del día mediante un conjunto K de vehículos que parten desde un centro de distribución (CD) y deben atender a un conjunto J de clientes. Cada cliente $j \in J$ presenta una demanda en peso p_j [kg], una demanda en volumen v_j [m³], un tiempo de servicio T_j [min].

Cada vehículo $k \in K$ posee una capacidad máxima en peso P_k [kg], y en volumen V_k [m³]. Además, cada camión incurre en un costo fijo c_k [\$] cuando es utilizado durante la jornada, y en un costo variable asociado a la distancia recorrida. Este último se calcula a partir del precio del combustible g [\$ / lt] y del rendimiento del vehículo o_k [km/lt].

Entre cada par de nodos (i, j) se conoce la distancia d_{ij} [km] y la velocidad promedio u_{ij} , lo que permite estimar el tiempo de desplazamiento entre clientes y el centro de distribución. El tiempo total asociado a una visita incluye tanto el tiempo de viaje como el tiempo de atención T_j [min]. requerido por cada cliente. Asimismo, se considera que por normativa una ruta no puede exceder 5 horas de operación.

5.2 Conjuntos, parámetros e índices

J : Conjunto de clientes.

J^C : Conjunto de nodos. $J^C \equiv J \cup \{0: CD\}$

K : Conjunto de camiones.

g : Costo por lt de combustible.

c_k : Costo por utilizar un camión.

p_j : Peso del pedido del cliente j en kg.

P_k : Peso máximo del camión k en kg.

v_j : Volumen del pedido del cliente j en m³.

V_k : Volumen máximo del camión k en m³.

o_k : Rendimiento del vehículo k en $\frac{km}{lt}$.

d_{ij} : Distancia entre cliente i y cliente j .

u_{ij} : Velocidad máxima entre cliente i y cliente j .

T_j : Tiempo que se demora en atender al cliente j .

j : Clientes. $j \in J^C \equiv J \cup \{0: CD\}$.

k : Camiones. $k \in \{1, 2, \dots, K\}$

5.3 Variables

Se tienen 3 variables en el modelo. El objetivo de x_{ijk} es determinar el orden de las rutas en cada camión. y_j permite ordenar en tiempo cada cliente en la ruta. Se asume que el cliente 0 (CD) tiene solo tiempo de llegada, el tiempo de salida del CD es 0. w_k sirve para desincentivar al modelo a utilizar más camiones de los necesarios.

$$x_{ijk} = \begin{cases} 1: & \text{Se atiende al cliente } j \text{ luego del cliente } i \text{ en el camión } k \\ 0: & \text{eoc.} \end{cases}$$

y_j : Tiempo en que se espera llegar a la ubicación del cliente j en min

$$w_k = \begin{cases} 1: & \text{El camión } k \text{ se utiliza} \\ 0 & \text{eoc.} \end{cases}$$

5.4 Restricciones y Función Objetivo

La función objetivo minimiza costos [CLP], de traslado y de utilizar camiones. Se considera que el traslado considera solamente el costo de la bencina. Cualquier costo de mantención de vehículos, tag, y otros costos variables en el trayecto debe cubrirlo la empresa. Asimismo, el costo de utilizar un camión se compone exclusivamente del costo del chofer contratado, el cual es de aproximadamente 40.000 pesos diarios.

$$\min \sum_{k=1}^K w_k c_k + \sum_{i \in J^C} \sum_{j \in J^C} \sum_{k=1}^K x_{ijk} d_{ij} \frac{g}{o_k}$$

Se consideran las siguientes restricciones lineales para el modelo:

R1[]: La cantidad de veces que un camión llega al CD es igual a la cantidad de veces que un camión sale del CD, igual a si se utiliza el camión.

$$\sum_{i \in J} x_{i0k} = \sum_{i \in J} x_{0ik} = w_k \quad \forall k \in \{1, 2, \dots, K\}$$

R2 [m^3]: Capacidad volumétrica máxima del camión.

$$\sum_{i \in J^C} \sum_{j \in J} x_{ijk} v_j \leq V_k \quad \forall k \in \{1, 2, \dots, K\}$$

R3 [*kg*]: Peso máximo de cada camión

$$\sum_{i \in J^c} \sum_{j \in J} x_{ijk} p_j \leq P_k \quad \forall k \in \{1, 2, \dots, K\}$$

R4 [*min*]: Los pedidos pueden entregarse como máximo el 5 horas después de su partida menos lo que se demora en volver al CD.

$$y_j \leq 5 * 60 - d_{j0}/u_{j0} - T_j \quad \forall j \in J$$

R5 [*min*]: Se atiende un pedido en al menos el tiempo que llega el anterior más el tiempo que se demora en atenderlo más el tiempo que demora en llegar de *i* a *j*. En caso de no utilizar ese arco con ese camión, desactivar restricción.

$$y_j \geq y_i + T_i + \frac{d_{ij}}{u_{ij}} - 300(1 - x_{ijk}) \quad \forall i \in J^c, j \in J, \forall k \in \{1, 2, \dots, K\}$$

R6 [*min*]: Se parte del CD en t_0 .

$$y_0 = 0$$

R7 $\square$: Se debe entrar a ubicación de cada cliente una vez.

$$\sum_{k=1}^K \sum_{i \in J^c} x_{ijk} = 1 \quad \forall j \in J$$

R8 $\square$: Se debe salir de la ubicación de cada cliente una vez.

$$\sum_{k=1}^K \sum_{i \in J^c} x_{jik} = 1 \quad \forall j \in J$$

R9 $\square$: Eliminación de autoatención.

$$x_{jjk} = 0 \quad \forall k \in \{1, 2, \dots, K\}, \forall j \in J^c$$

R10 $\square$ y [*min*]: Naturaleza.

$$x_{ijk} \in \{0, 1\} \quad \forall k \in \{1, 2, \dots, K\}, \forall i \in J^c, \forall j \in J^c$$

$$y_j \geq 0 \quad \forall j \in J^c$$

$$w_k \in \{0, 1\} \quad \forall k \in \{1, 2, \dots, K\}$$

5.5 Simplificaciones

La idea del modelo era hacer la mayor cantidad de ajustes postoptimales y antes de ejecutar el modelo, debido a que cada implementación en el modelo de cualquier característica considera una

aplicación de tiempo y una reducción de clientes con una complejidad algorítmica considerable, por lo que se redujo al mínimo las características. Estas características mínimas fueron:

- VRP: Atención a una cantidad de clientes con una cantidad de camiones desde un solo CD.
- Limitaciones de carga: Cada camión tiene un máximo de peso y volumen.
- 5 horas de conducción máximas: Por límites legales cada conductor puede conducir máximo 5 horas seguidas. (Si bien considerando las paradas no son 5 horas de solo conducción, se consideran para poder hacer 2 turnos como se verá más adelante) Se aplica como ventanas de tiempo a los camiones.
- Uso de distancias y velocidades de las calles reales.
- Flota homogénea.
- Tiempo de servicio variable en función del peso del pedido en kilos x , obtenido con la siguiente fórmula: $t(x) = \frac{27}{\ln(800)} \ln(x + 1) + 3$, donde $t(x)$ se mide en minutos. Se ocupa una distribución logarítmica para reflejar las economías de escala en el tiempo de despacho, con un mínimo de 3 minutos con 0 kilos y 30 minutos con 799 kg.

Las que se consideraron preoptimización:

- Obtención de coordenadas por geocodificación
- Limpieza de datos
- Reorden de la carga diaria de pedidos
- Obtención del costo de bencina en función de la API (<https://api.cne.cl>, <https://beta.api.cne.cl>, https://api.bencinaenlinea.cl/api/estaciones/precios_combustibles/reporte_zonal),

Las consideraciones que fueron aplicadas postoptimalmente fueron las siguientes:

- Rutas reales
- Tiempos de viaje aproximados en función del horario con una matriz de tiempos y comunas.
- Reasignación de rutas a camiones en varios turnos, con una diferencia de 30 min entre cada salida.
- Merge de rutas con un solo cliente (Problema de la heurística utilizada).
- Aplicación de local search para mejorar el óptimo
- Aplicación de heurísticas como Tabú Search para mejorar el óptimo
- Asignación de camiones considerando camiones máximos y ventanas de tiempo diarias.

Las consideraciones que no se tomaron en cuenta fueron:

- Cliente pueda solicitar la ventana horaria de su visita.
- Velocidades exactas por tramo en función de la hora. (Se usaron aproximaciones)
- Incertidumbre en las rutas.

5.6 Reflexión del modelo

El modelo inicial logra capturar de manera consistente la lógica esencial del problema de distribución, pues incorpora los elementos estructurales mínimos para construir rutas operativamente viables y económicamente comparables. En este sentido, su nivel de realismo es adecuado como punto de partida, ya que representa de forma razonable las restricciones físicas, temporales y de costo que enfrenta la operación. Sin embargo, este realismo fue deliberadamente acotado para preservar trazabilidad computacional y permitir una implementación funcional sobre datos reales.

En consecuencia, se omitieron componentes importantes del sistema, no por falta de relevancia, sino por el costo algorítmico que implicaba incorporarlos directamente al modelo base. Estas dimensiones fueron simplificadas o desplazadas hacia etapas pre y postóptimas con el fin de evitar un crecimiento excesivo en complejidad, tiempos de resolución y pérdida de escalabilidad. Así, más que una representación exhaustiva de la realidad, el modelo debe entenderse como una aproximación estratégica suficientemente realista para orientar decisiones logísticas, pero lo bastante simple para seguir siendo resoluble y útil en la práctica.

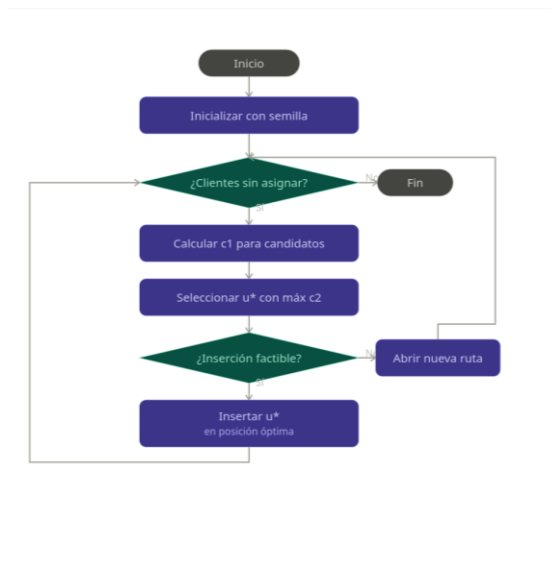
6 Desarrollo del Algoritmo

El desarrollo del algoritmo se basó en una heurística, ya que el problema de ruteo con restricciones reales tuvo limitaciones en 60 clientes con métodos exactos, por lo que no era suficientemente escalable y se tuvo que utilizar una heurística. Por esta razón, se optó por construir una solución inicial con una heurística de inserción tipo Solomon I1 y luego mejorarla con procedimientos de búsqueda local y búsqueda tabú. Esta nació de utilizar las principales heurísticas adaptadas al problema, junto con metaheurísticas ejecutando todas las combinaciones de ellas, y la configuración anteriormente mencionada fue la mejor. Las siguientes fueron las heurísticas y metaheurísticas utilizadas:

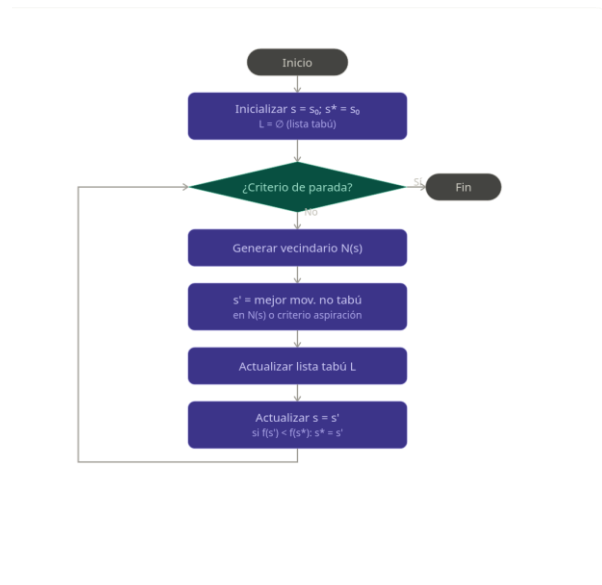
- Solomon I1
- Clarke and Wright
- MultiStart
- Regret2Parallel
- ALNS
- SweepGM74
- Tabú Search
- Simulated Annealing
- Genetic Algorithm

Se detallan los algoritmos en los siguientes diagramas de flujo de manera sencilla, junto con el pseudocódigo completod de solomón I1.

Solomon I1



Tabú search



Pseudocódigo ALGORITMO SOLOMON_I1 (VRPTW)

Entrada:

C = clientes no atendidos

0 = depósito

$K_{\max}$ = máximo número de vehículos/rutas

Q = capacidad del vehículo (peso/volumen)

$[e_i, l_i]$ = ventana de tiempo de cada cliente i

s_i = tiempo de servicio

$d(i,j), t(i,j)$ = distancia y tiempo de viaje

Parámetros: $\alpha_1, \alpha_2, \lambda$

Salida:

Conjunto de rutas factibles R

1. $R \leftarrow \emptyset$
2. $U \leftarrow C$ // clientes no enrutados
3. Mientras $U \neq \emptyset$:
4. Si $|R| = K_{\max}$:
5. retornar INFECTIBLE (o solución parcial + reparación)
6. // Selección de cliente semilla
7. $seed \leftarrow \arg\max_{\{u \in U\}} d(0,u)$ // típico en I1
8. $r \leftarrow [0, seed, 0]$
9. $U \leftarrow U \setminus \{seed\}$
10. calcular horarios de r (arribos, esperas, inicio servicio)

11. Repetir:
12. $\text{mejor_global} \leftarrow \text{NONE}$
13. Para cada $u \in U$:
14. $\text{mejor_u} \leftarrow \text{NONE}$
15. Para cada arco consecutivo (i,j) en r :
16. $r' \leftarrow \text{insertar } u \text{ entre } i \text{ y } j \text{ en } r$
17. Si r' es factible (capacidad + ventanas de tiempo + duración):
18. $c11 \leftarrow d(i,u) + d(u,j) - d(i,j)$ // costo espacial
19. $c12 \leftarrow \text{fin_ruta}(r') - \text{fin_ruta}(r)$ // impacto temporal
20. $c1 \leftarrow \alpha_1 * c11 + \alpha_2 * c12$ // costo de inserción
21. Si $\text{mejor_u} = \text{NONE}$ o $c1 < \text{mejor_u}.c1$:
22. $\text{mejor_u} \leftarrow (\text{pos}=(i,j), c1)$
23. Si $\text{mejor_u} \neq \text{NONE}$:
24. $c2 \leftarrow \lambda * d(0,u) - \text{mejor_u}.c1$ // criterio II
25. Si $\text{mejor_global} = \text{NONE}$ o $c2 > \text{mejor_global}.c2$:
26. $\text{mejor_global} \leftarrow (\text{cliente}=u, \text{pos}=\text{mejor_u}.pos, c2)$
27. Si $\text{mejor_global} = \text{NONE}$:
28. romper // no hay más inserciones factibles en esta ruta
29. Insertar $\text{mejor_global}.cliente$ en $\text{mejor_global}.pos$ dentro de r
30. $U \leftarrow U \setminus \{\text{mejor_global}.cliente\}$
31. actualizar horarios de r
32. Añadir r a R
33. Retornar R

Otra implementación fue el uso de knearest para reducir la complejidad algorítmica, bajo el supuesto de que difícilmente un camión va a viajar a otro que esté mucho más lejano que sus vecinos. Esto se implementó para hacer rutas reales de manera eficaz, ya que se aplica dijkstra

para encontrar los k clientes más cercanos y sus distancias reales, y A^* para calcular la distancia al CD.

Entre las principales decisiones técnicas estuvo priorizar un enfoque práctico por sobre uno completamente exacto. También se incorporaron elementos que acercan el modelo a una operación más realista, como las restricciones de capacidad, la duración máxima de las rutas y la estimación de costos. Otra decisión importante fue dejar algunas validaciones antes de optimizar y algunos ajustes después de obtener la solución, lo que ayudó a mejorar la eficiencia e interpretabilidad del modelo. Como se dijo antes, se priorizó el desarrollo realista utilizando backend, frontend, almacenamiento en una base de datos relacional y los algoritmos se almacenaron en una carpeta en el backend.

Los principales aciertos fueron el testeo exhaustivo en diversas instancias de prueba, para asegurar la factibilidad y efectividad de las soluciones, además de enfocarse siempre en que el usuario estuviera cómodo con las implementaciones, mediante integración de dashboards, mapas con las rutas para seleccionar, entre otras.

Los desaciertos fueron en primer lugar, utilizar la librería gurobipy para las pruebas iniciales, que eran las que sabían usar los integrantes y la universidad brinda. Sin embargo, estas no se pueden utilizar debido a que no son gratuitas ni open source. En segundo lugar, el dejar corriendo tabú search durante horas cuando este se estanca en soluciones a los 20 segundos, y no mejora desde ahí.

```

Capstone/
|
├─ app/                                # Código fuente de la aplicación
|   ├── main.py                        # Punto de entrada del frontend (PyQt6)
|   ├── qt_runtime.py                  # Configuración del runtime Qt
|   ├── validation_heuristics.py       # solomon_hard_fleet + HARD_FLEET_PENALTIES
|   ├── validation_report.py           # Generador de informe de validación del modelo
|   |
|   ├── backend/                       # Servidor FastAPI (puerto 8000)
|   |   ├── main.py                   # Punto de entrada del backend
|   |   ├── api/
|   |   |   ├── router.py              # Todos los endpoints REST + loop multi-día
|   |   |   ├── models/                # Modelos de optimización VRP
|   |   |   |   ├── routing/
|   |   |   |   |   ├── heuristics.py    # Clarke-Wright savings + ALNS
|   |   |   |   |   ├── literature_heuristics.py # Solomon I1 (heurística activa)
|   |   |   |   |   └─ metaheuristics.py  # Tabu Search (mejora activa)
|   |   |   |   ├── Modelo*.py          # Formulaciones exactas con Gurobi (requieren licencia)
|   |   |   |   └─ SoluciónHeurística.py
|   |   |   ├── schemas/
|   |   |   |   └─ optimization.py       # OptimizerParams + schemas de auth
|   |   |   └─ services/                 # Lógica de negocio
|   |   |       ├── auth_service.py
|   |   |       ├── cleaning_service.py
|   |   |       ├── data_service.py
|   |   |       ├── db_persistence.py
|   |   |       ├── fuel_price_service.py
|   |   |       ├── job_store.py
|   |   |       ├── optimizer_service.py
|   |   |       ├── road_routing.py
|   |   |       └─ traffic_factor_dataset_builder.py
|   |
|   └─ config/
|       ├── settings.py                 # Lee .env y expone configuración global
|       ├── .env                        # Variables de entorno (NO se sube a git)
|       └─ .env.example                 # Plantilla de variables de entorno
|
├─ database/
|   ├── connection.py                  # Engine SQLAlchemy + get_session()
|   ├── models.py                      # Modelos ORM (Usuario, Producto, Venta, Detalle)
|   |   └─ repositories/               # CRUD por tabla
|   |       ├── base_repository.py
|   |       ├── usuario_repository.py
|   |       ├── producto_repository.py
|   |       └─ venta_repository.py

```

```

| |     └─ detalle_repository.py
| |
| | └─ frontend/                # Interfaz de usuario PyQt6
| |   └─ app.py                 # QMainWindow + navegación entre pantallas
| |   └─ views/
| |     └─ login_view.py        # Login, registro y reset de contraseña
| |     └─ main_view.py         # Workspace principal (Data Hub, Optimizador, Resultados)
| |   └─ widgets/
| |     └─ components.py        # Componentes Qt reutilizables
| |   └─ resources/
| |     └─ styles/
| |   └─ workers/               # QThreads para operaciones largas
| |     └─ health_worker.py
| |     └─ upload_worker.py
| |     └─ data_hub_worker.py
| |     └─ progress_worker.py
| |     └─ request_worker.py
| |
| | └─ __tests__/               # Suite de pruebas
| |   └─ conftest.py
| |   └─ unit/
| |   └─ integration/
| |   └─ stress/
| |   └─ results/               # Reportes generados por pytest y locust
| |
| └─ RAW_DATA/                  # Datasets de ejemplo (datos ficticios de Santiago)
| └─ CLEAN_DATA/                # Versiones pre-limpias de los datasets
| └─ SCRIPTS/                   # Notebooks y scripts de exploración
| └─ test_cases/                # Exportaciones CSV de corridas de prueba (informe técnico)
| └─ cache/                     # Caché de resultados OSRM (generado automáticamente)
| └─ geocache.json              # Caché persistente de geocodificación Nominatim
| └─ TODO.md                     # Deuda técnica y trabajo pendiente

```

7 Incorporación de Restricciones Reales

Como se mencionó anteriormente, se incorporaron las restricciones reales mínimas para conseguir una representación realista del modelo.

- Volumen y peso: Se modeló el volumen y el peso en función de los vehículos escogidos, considerando un 10% de holgura para espacios muertos. Se consideró una flota homogénea de los siguientes vehículos:

Este camión podría ser el KYCX5 plus 2.0 CARGO BOX, con una capacidad de 1.850 Kg, y un volumen de carga de 3585 / 1770 / 1550 (Largo/ancho/alto) 9,8m³ y un precio de aproximadamente \$13.790.000 sin IVA.

- Tiempos de servicio: En cuanto a los tiempos de servicio, se utilizó una aproximación logarítmica para reflejar las economías de escala en el tiempo de despacho, con un mínimo de 3 minutos con 0 kilos y 30 minutos con 799 kg.
- Embalaje: Se consideró que el embalaje de todo dura 30 min, por lo que fuera del modelo la carga y embalaje se debe realizar previo a la salida de cada camión, los cuales cada uno tiene una holgura de 30 min.

El modelo pasó de una lógica de ruteo limpia a una representación operativa más fiel. Primero, incorporar simultáneamente peso y volumen (con 10% de holgura por espacios muertos) transforma la capacidad en una restricción física real, no solo teórica. Luego, usar una función logarítmica de tiempo de servicio introduce comportamiento realista en terreno (economías de escala en despacho). Explicitar el embalaje/carga previo como condición operativa agrega contexto de ejecución, aunque quede fuera del núcleo matemático del ruteo.

Estas decisiones suelen volver el problema más exigente y, por tanto, más creíble: aparecen menos soluciones factibles y sube la sensibilidad a la composición de pedidos por ruta. También mejora la calidad práctica de la planificación, porque evita rutas “bonitas en papel” pero inviables por capacidad o tiempos reales de atención. El punto a cuidar es que el embalaje esté fuera del modelo: eso simplifica, pero puede ocultar cuellos de botella, como siguiente evolución, convendría modelar explícitamente esa capacidad de preparación en depósito.

8 Modelación del Embalaje, Volumen y Peso

En el código, la modelación de volumen y peso se incorporó como una restricción de capacidad dual por ruta: cada cliente aporta una demanda de peso p y volumen v , y cada camión tiene límites máximos que no se pueden sobrepasar P y V . Esto se verifica directamente en la factibilidad de ruta y también en la evaluación de soluciones, donde se calculan violaciones específicas de peso y volumen. Además, el tiempo de servicio por cliente se modela con una función logarítmica en función del peso, lo que hace que el modelo sea más realista que uno con tiempos constantes, porque representa economías de escala en la atención de pedidos más grandes.

9 Resultados Obtenidos

Los resultados experimentales se obtuvieron sobre un conjunto de datos real de órdenes de despacho correspondientes a la Región Metropolitana de Chile, conformado por 4.180 pedidos distribuidos en múltiples semanas calendario. De este total, 2.740 órdenes (65,55%) contaban con direcciones geocodificables válidas y fueron procesadas por el optimizador. Las 1.440 órdenes restantes presentaban direcciones inexistentes, incompletas o no reconocibles por el geocodificador, por lo que fueron excluidas del pipeline de ruteo en la etapa de preprocesamiento.

Configuración base. La configuración de referencia utilizó Tabu Search con un presupuesto de 20 segundos por jornada, un límite de 150 entregas por día y sin restricción de capacidad volumétrica explícita. Bajo esta configuración, el sistema procesó la totalidad del horizonte de planificación en 35 días, alcanzando un costo total de \$9.445.706 CLP (promedio de \$269.877 CLP/día) con un pico de 13 vehículos simultáneos en el día de mayor demanda. El tiempo de ejecución total fue de 734,71 segundos, de los cuales 690,29 s correspondieron al solver (generación de matriz + Solomon II + Tabu Search).

Análisis de sensibilidad sobre el presupuesto Tabu Search. Se evaluó el impacto del tiempo de búsqueda asignado a la metaheurística, manteniendo fijos el resto de los parámetros.

Configuración	Costo total (CLP)	Camiones pico	Tiempo ejecución (s)
Sin Tabu Search	9.862.803	14	57
Tabu 5 s	9.632.243	14	249
Tabu 10 s	9.555.134	13	409
Tabu 20 s (base)	9.445.706	13	735
Tabu 30 s	9.370.081	13	1.044
Tabu 60 s	9.342.584	12	1.933

Tabla 1. Resultados según presupuesto de Tabu Search.

La reducción de costo respecto al caso sin Tabu Search fue de 4,2% para el presupuesto de 20 s y de 5,3% para 60 s, con una relación de ahorro marginal decreciente: cada 10 s adicionales de búsqueda generan retornos sucesivamente menores. Los 20 s representan el punto de inflexión donde se captura el 80% de la mejora máxima observada al 38% del costo computacional del escenario de 60 s.

Análisis de sensibilidad sobre la restricción de flota. Se evaluaron límites duros de 10 y 15 vehículos frente a la configuración base sin restricción vinculante.

Límite de flota	Costo total (CLP)	Camiones pico	Días procesados
10 camiones	10.126.245	10	37
Sin límite (base)	9.445.706	13	35
15 camiones	9.416.877	13	35

Tabla 2. Resultados según límite de flota.

El límite de 15 vehículos resultó no vinculante: el optimizador nunca demandó más de 13 camiones de forma natural. La restricción de 10 vehículos, en cambio, activó el mecanismo de carry-over, extendiendo el horizonte de operación en 2 días y elevando el costo total en 7,2%.

Análisis de sensibilidad sobre la capacidad volumétrica. Se compararon configuraciones de 7 m³ y 12 m³ por vehículo (con holgura del 10% por espacios muertos) frente al caso base sin restricción.

Capacidad por vehículo	Costo total (CLP)	Camiones pico	Días procesados
7 m ³	9.869.631	12	35
Sin restricción (base)	9.445.706	13	35
12 m ³	9.133.701	13	35

Tabla 3. Resultados según capacidad volumétrica por vehículo.

La configuración de 12 m³ redujo el costo total en 3,3% respecto a la base, mientras que la de 7 m³ lo incrementó en 4,5% pese a utilizar un vehículo menos en pico. El mecanismo subyacente es la acumulación de costos fijos por viaje: una menor capacidad fuerza más retornos al centro de distribución por jornada, elevando el costo operacional agregado independientemente del conteo de vehículos.

Análisis de sensibilidad sobre el parámetro deliveries_per_day. Se evaluó el efecto de limitar la cantidad de órdenes asignadas por jornada.

Máximo entregas / día	Costo total (CLP)	Camiones pico	Días procesados
100 entregas / día	8.907.684	8	43
150 entregas / día (base)	9.445.706	13	35
200 entregas / día	9.320.518	13	35

Tabla 4. Resultados según límite de entregas por día.

Restringir a 100 entregas diarias redujo el costo total en 5,7% y el pico de flota de 13 a 8 vehículos, al costo de 8 días adicionales de operación. Este resultado refleja que instancias de menor tamaño diario permiten al algoritmo construir rutas de mayor calidad relativa. El límite de 200 entregas por día resultó no vinculante dado que la distribución natural de la demanda en el dataset raramente supera las 150 órdenes por fecha.

10 Validación del Modelo

El proceso de validación del modelo de ruteo de vehículos con ventanas de tiempo (VRPTW) se concibió bajo un enfoque determinista mediante pruebas de estrés computacional y verificación rigurosa de coherencia operativa. El objetivo principal fue evaluar la robustez, el tiempo computacional y la capacidad de las heurísticas para generar soluciones factibles de alta aplicabilidad industrial frente a escalas de demanda variables.

Para asegurar que las pruebas estuviesen aisladas de latencias de red y cuellos de botella externos (originados por el contenedor local OSRM), se prescindió transitoriamente de la conectividad vial real. El costo de las trayectorias se simuló empleando distancias geométricas ortodrómicas (fórmula de Haversine), asumiendo una velocidad promedio base de la flota.

10.1 Pruebas realizadas, sus propósitos y sus resultados

Se generaron conjuntos de clientes pseudoaleatorios dispersos bajo coordenadas geolocalizadas realistas de la Región Metropolitana. Para cada escenario, el sistema resolvió la demanda mediante un ensamblaje dual: una **Construcción Heurística Base** (Solomon II) para asegurar viabilidad técnica temprana, seguida de una **Optimización y Búsqueda Local Estocástica** (Tabu Search) restringida a 20 segundos de procesamiento profundo, con un mecanismo de *fallback* controlado en caso de caer en óptimos locales deficientes.

- **TC-01 (Operación Diaria Estándar)**
 - **Propósito:** Simular el comportamiento basal de la jornada, concentrando una demanda de 100 nodos en un radio de ~ 3 km, asumiendo una utilización media de la capacidad en peso cercana al 80%.
 - **Resultados:** El modelo logró una cobertura del 100% (0/100 nodos no entregados). Utilizó de manera eficiente 9 de los 10 vehículos disponibles. La ruta más larga se ajustó milimétricamente a la jornada, alcanzando los 299 minutos.
- **TC-02 (Operación Dispersa)**
 - **Propósito:** Medir el efecto de la dispersión espacial proyectando 60 nodos distribuidos hasta a ± 20 km desde el Centro de Distribución, forzando la creación agresiva de clústeres.
 - **Resultados:** Cobertura del 100% (0/60 nodos no entregados). El algoritmo agrupó exitosamente la demanda utilizando 8 de los 10 camiones, con una duración máxima de ruta que topó exactamente el límite de 300 minutos.
- **TC-03 (Alta Demanda / Stress Test)**
 - **Propósito:** Modelar la saturación de la flota (ej. eventos Cyber o Navidad), escalando a 280 nodos confinados a ± 15 km y exigiendo despachos a un umbral límite del 91% respecto a la capacidad de carga nominal.

- **Resultados:** Saturación completa de la flota (20/20 vehículos utilizados). Dada la restricción física de la capacidad, 65 de 280 nodos (23.2%) quedaron sin asignar. La ruta más larga alcanzó los 298 minutos.
- **TC-04 (Límite Computacional de Alta Carga)**
 - **Propósito:** Estresar la complejidad temporal algorítmica y la integridad de memoria matricial distribuyendo masivamente 800 peticiones en un radio de ± 22 km, con un límite de 60 camiones de alta capacidad.
 - **Resultados:** Utilización total de la flota (60/60 vehículos). Ante el desborde operacional virtual, el modelo dejó 266 de 800 nodos (33.2%) sin servir para mantener la factibilidad del resto de la operación. El límite de tiempo de conducción se respetó estrictamente (300 minutos máximo).

10.2 Coherencia de resultados

Independiente del desempeño en minimización de distancias, se validaron programáticamente cinco reglas de negocio (axiomas) sobre los arreglos matriciales de salida para garantizar la integridad lógico-operativa estructural del árbol de rutas.

1. Regla 1: Restricción de capacidad dura (VERIFICADA)

- a. Para cada ruta producida, se validó que la suma de las demandas en peso de los nodos asignados no excede la capacidad máxima del vehículo (P), comparando con una tolerancia numérica de 10^{-6} kg. La capacidad actuó como restricción dura en la construcción Solomon (solo permitiendo inserciones factibles) y como una penalidad prohibitiva en Tabu Search (5,000 por kg de exceso).

2. Regla 2: Integridad de la solución y unicidad (VERIFICADA)

- a. Se confirmó la ausencia total de nodos duplicados en el flujo resolutivo y que todos los nodos visitados pertenecen al conjunto original de clientes válidos. La cobertura de demanda se abordó intencionalmente como una restricción blanda (penalidad de 50,000 por nodo no entregado). Esto incentiva la asertividad máxima sin generar cuellos irresolubles o fallas del sistema en instancias físicamente inviables (como se evidenció en los casos TC-03 y TC-04).

3. Regla 3: Prevención de grafo aislado / Subtours (VERIFICADA)

- a. Se comprobó que cada ruta con al menos un cliente comienza y termina inequívocamente en el nodo depósito (índice 0). Las soluciones de Solomon II preservan esta conectividad por construcción, y los movimientos de vecindad del Tabu Search la mantienen en todo momento gracias a la función `repair_solution_vrptw`, descartando ciclos cerrados operacionalmente inválidos.

4. Regla 4: Dimensionamiento finito de flota (VERIFICADA)

- a. El número de rutas activas en la solución final jamás superó el parque de vehículos instanciado (K). Esto demostró la imposibilidad sistémica de sobregirar la flota,

operando como límite duro en Solomon y con una penalidad extrema de 1,000,000 por ruta adicional en el motor Tabu Search.

5. Regla 5: Acotamiento temporal y jornada laboral (VERIFICADA)

- a. Ninguna ruta superó el límite operativo de 300 minutos (5 horas). Este límite actuó como restricción estricta en las validaciones de Solomon y como penalidad prohibitiva (5,000 por minuto excedido) en la metaheurística. En las iteraciones finales, el sistema logró acercamientos asintóticos perfectos (298 - 300 minutos) exprimiendo la capacidad sin cometer infracciones.

10.3 Comparación con expectativas

Los resultados obtenidos se alinean de manera precisa con el comportamiento teórico y operacional esperado del modelo bajo distintos niveles de estrés.

En escenarios de demanda normal a moderada (TC-01 y TC-02), se esperaba que el algoritmo lograra una cobertura total de los clientes minimizando el uso de la flota. Esto se cumplió exitosamente, dejando vehículos en reserva y construyendo rutas que respetaron holgadamente los límites de peso, al mismo tiempo que maximizaron el uso de la ventana de 5 horas de conducción.

Por otro lado, frente a las pruebas de estrés extremo (TC-03 y TC-04), la expectativa fundamental no era que el modelo lograra despachar el 100% de la carga, lo cual era matemáticamente imposible dada la suma de demandas versus la capacidad volumétrica y temporal instalada, sino que fallara de manera controlada e inteligente. El sistema cumplió esta expectativa a cabalidad: en lugar de arrojar errores computacionales, generar rutas infactibles (subtours) o violar la jornada laboral de los conductores, el algoritmo saturó la flota y el tiempo disponible al máximo legal, dejando sin asignar únicamente la porción de demanda excedente (23.2% y 33.2% respectivamente).

Esto demuestra que el esquema de penalidades de la metaheurística Tabu Search y las restricciones duras de Solomon II son lo suficientemente robustos para priorizar siempre la viabilidad técnica y legal de la operación por sobre la cobertura irreal, validando el modelo para su aplicación en un entorno logístico industrial.

11 Análisis Crítico del Proceso de Desarrollo del Proyecto

11.1 Aciertos Metodológicos y Técnicos

- **Desacoplamiento de Datos y Parametrización Dinámica:** Una fortaleza central del diseño fue la total separación entre la lógica del algoritmo de ruteo y los datos de entrada. En lugar de incrustar (*hardcodear*) parámetros operativos como constantes dentro del código, el sistema fue diseñado para recibir dinámicamente variables como las capacidades vehiculares, los rendimientos de combustible y los volúmenes de carga máxima. Esta abstracción fue fundamental para realizar pruebas de estrés escalables y permitir simulaciones profundas bajo distintos escenarios logísticos.
- **Arquitectura Cliente-Servidor (Frontend/Backend Segregados):** La implementación temprana de una arquitectura distribuida, comunicada exclusivamente mediante un protocolo REST API, resultó ser una decisión crítica para la estabilidad del sistema. Dado que el modelo matemático (especialmente durante las iteraciones de la metaheurística Tabu Search en instancias masivas) genera un alto consumo de CPU, la segregación evitó el congelamiento de la interfaz de usuario (PyQt6), permitiendo una experiencia de monitoreo asíncrona fluida mientras el backend procesaba en segundo plano.

11.2 Desaciertos y Oportunidades de Mejora

- **Subestimación del Trade-off Tiempo vs. Calidad (Costo Operativo):** Durante las fases iniciales del desarrollo, se priorizó erróneamente la velocidad de convergencia algorítmica por sobre la minimización absoluta de la función objetivo. La validación posterior demostró que, en un entorno logístico industrial, sacrificar un porcentaje marginal en la calidad de la solución (ej. un aumento del 0.11% en el costo total de la ruta) para ganar unos segundos de ejecución representa una decisión anti-económica. En flotas a gran escala, esa fracción porcentual equivale a un alto impacto financiero, evidenciando un desajuste inicial entre el criterio matemático y la expectativa real del negocio.
- **Falta de Validación Temprana de Submodelos:** Un error metodológico fue no aislar y someter a pruebas unitarias rigurosas cada componente algorítmico durante la fase de desarrollo temprano. Esto provocó que ciertas imprecisiones (como asunciones métricas incorrectas) se propagaran hacia las etapas de integración, generando resultados temporalmente tergiversados (falsos positivos en eficiencia). Este sesgo obligó a realizar refactorizaciones no planificadas en etapas avanzadas para normalizar los resultados hacia métricas operativas reales.

11.3 Decisiones Clave que Definieron el Proyecto

- **Pivote de Arquitectura: De Solución *Ad-hoc* a Modelo SaaS:** La decisión estratégica más disruptiva fue abandonar el enfoque de construir un modelo aislado y hecho a la medida de un solo cliente, pivotando hacia una arquitectura orientada a Software as a

Service (SaaS). Esto forzó al equipo a abstraer el "contexto del cliente", diseñando un motor VRP agnóstico y generalizable que, sustentado en la parametrización dinámica y el aislamiento de bases de datos por usuario, garantiza la escalabilidad del producto hacia múltiples cuentas.

- **Hibridación Algorítmica (Heurística + Metaheurística):** Desde la perspectiva de investigación de operaciones, el rechazo a usar un único método de resolución cambió el rumbo del desempeño técnico. La decisión de ensamblar un pipeline dual, utilizando una heurística constructiva veloz (tipo Solomon II) para garantizar una solución semilla inicial factible, e inyectándola inmediatamente en un motor de Búsqueda Tabú para escapar de óptimos locales, fue el factor decisivo que permitió al sistema alcanzar niveles de minimización de distancia viables a nivel industrial sin colapsar por uso de memoria.

12 Lecciones Aprendidas

Las lecciones aprendidas, por integrante  del grupo, son:

Joaquín Parraud:

Las principales lecciones aprendidas a lo largo de este proyecto incluyen el uso de REST API, testing continuo de servicios y la creación efectiva de instancias de prueba para comprobar los KPI de modelos de optimización heurística. Además, la necesidad de implementación y mezcla de soluciones no encontradas en la literatura para el servicio de optimización de rutas. Otra lección fue el uso de servicios locales open source (OSRM, Nominatim) y su inserción en el diseño de la solución.

Sebastián Ramírez:

Durante el desarrollo me encargué principalmente del desarrollo del modelo, la configuración de heurísticas e integración de ellas dentro de la aplicación. Mis principales aprendizajes fueron la terminología y taxonomía de VRPs, la configuración en un entorno real, el desarrollo e interpretación de instancias de prueba, desarrollo de KPIs y puse en práctica mis habilidades en modelamiento matemático. También debo destacar el análisis y pensamiento crítico por los paper que tuve que crear (Algunos de los referenciados debajo). También la configuración real de parámetros y búsqueda exhaustiva de maneras de representar la realidad no dentro del modelo, si no postóptimalmente.

Vicente Ramírez:

Lo principal aprendido es el uso de herramientas locales a través de distintos motores, tales como docker. Ello viene con un trade-off: Ejecución más rápida y eficiente de la geocodificación, pero es necesario poner esfuerzo extra en el setup de la base de datos local, además del uso de recursos y almacenamiento de los equipos propios. Otra lección aprendida es el uso de un equipo propio como alojamiento de una base de datos a la que se pueden conectar otros equipos, a través de permisos específicos entregados por el administrador.

13 Limitaciones del Trabajo

A pesar de la robustez de la solución propuesta, el presente trabajo se delimitó bajo ciertos supuestos operacionales que restringen su aplicación inmediata en entornos altamente dinámicos. Las principales limitaciones identificadas son:

- **Modelamiento de Variables Dinámicas:** El algoritmo actual asume condiciones de tránsito estáticas. La integración de APIs de tráfico en tiempo real y la gestión dinámica de ventanas de tiempo (re-ruteo en ejecución) quedan fuera del alcance de esta iteración.
- **Restricción de depósitos:** El sistema actual no considera múltiples depósitos, restringiendo al usuario a usar un único depósito central.
- **Aprendizaje Histórico:** El sistema opera bajo un enfoque determinista de instancia única. No incorpora mecanismos de machine learning para utilizar el feedback de rutas históricas (tiempos reales de atención o zonas de congestión recurrente) en la calibración de futuras optimizaciones.
- **Experiencia y Gestión de Usuario:** Aunque el sistema cumple con su objetivo basal, la capa de presentación (DMS) carece de herramientas avanzadas de manipulación visual interactiva para el operador, así como de módulos analíticos predictivos para estimar fluctuaciones futuras en la demanda logística del cliente.

14 Trabajo Futuro.

A partir de las limitaciones identificadas y los resultados obtenidos, se proponen las siguientes líneas de extensión:

- Integración de tráfico dinámico en tiempo real: La matriz de distancias y tiempos actualmente se construye con velocidades estáticas (Haversine o OSRM sin perfil horario). La incorporación de APIs de tráfico en tiempo real permitiría capturar congestión recurrente por zona y hora, refinando los tiempos de llegada estimados y reduciendo incumplimientos de ventanas horarias en operación real.
- Soporte de múltiples depósitos: El modelo actual asume un único centro de distribución. La extensión al problema de múltiples depósitos (MDVRPTW) permitiría modelar redes de distribución con bodegas descentralizadas o cross-docking, aumentando significativamente el rango de aplicabilidad del sistema.
- Ventanas de tiempo por cliente: La formulación actual no permite que los clientes especifiquen franjas horarias de recepción. Incorporar ventanas de tiempo duras a nivel de cliente transformaría el modelo en un VRPTW completo, más alineado con las exigencias contractuales de grandes receptores (retail, industria).
- Calibración automática mediante aprendizaje histórico: El tiempo de servicio actualmente se estima con una función logarítmica fija del peso. El registro de tiempos reales de atención por cliente, zona o tipo de producto permitiría calibrar este parámetro de forma adaptativa, mejorando la precisión de los horarios estimados en las rutas generadas.
- Re-ruteo dinámico (online VRP): El sistema opera bajo un paradigma estático: las rutas se generan al inicio de la jornada y no se modifican. La integración de un módulo de re-ruteo ante eventos en tiempo real (cancelaciones, retrasos, pedidos de último minuto) representaría la evolución natural hacia un sistema de gestión logística reactivo.
- Análisis predictivo de demanda: Incorporar modelos de forecasting sobre la demanda diaria permitiría anticipar jornadas de alta carga y ajustar proactivamente la configuración de flota y el parámetro `deliveries_per_day`, reduciendo la dependencia de la configuración manual por parte del operador.

15 Conclusiones

El presente trabajo demostró la viabilidad técnica y operacional de implementar un optimizador de rutas CVRPTW sobre datos reales de distribución urbana en la Región Metropolitana, dentro de un sistema integrado que encapsula el pipeline completo de preprocesamiento, resolución y visualización. La hibridación Solomon II + Tabu Search produjo soluciones con una reducción de costos de 4,2% respecto a la construcción heurística pura con un presupuesto de 20 segundos de búsqueda, manteniendo la factibilidad respecto a todas las restricciones operacionales del modelo (capacidad, jornada laboral, conservación de flujo).

La decisión de desacoplar la complejidad del modelo central mediante etapas pre y postoptimales, geocodificación, limpieza, carry-over, reasignación multiturnos, permitió mantener el núcleo algorítmico resoluble en tiempos compatibles con operación diaria, sin sacrificar representatividad operacional. La arquitectura SaaS con aislamiento de datos por usuario demostró que este enfoque es generalizable a múltiples clientes sin degradación de rendimiento inter-instancia.

Las pruebas de estrés (TC-03, TC-04) validaron que el sistema falla de forma controlada ante demanda estructuralmente inviable: satura la flota y el tiempo disponible al máximo legal sin generar rutas infactibles ni subtours, comportamiento requerido para aplicación en entornos logísticos industriales.



16 Referencias

- Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91. <https://doi.org/10.1287/mnsc.6.1.80>
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265. <https://doi.org/10.1287/opre.35.2.254>
- Laporte, G. (2009). Fifty years of vehicle routing. *Transportation Science*, 43(4), 408–416. <https://doi.org/10.1287/trsc.1090.0301>
- Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle routing: Problems, methods, and applications* (2nd ed.). Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611973594>
- Bräysy, O., & Gendreau, M. (2005). Vehicle routing problem with time windows, part I: Route construction and local search algorithms. *Transportation Science*, 39(1), 104–118. <https://doi.org/10.1287/trsc.1030.0056>
- Bräysy, O., & Gendreau, M. (2005). Vehicle routing problem with time windows, part II: Metaheuristics. *Transportation Science*, 39(1), 119–139. <https://doi.org/10.1287/trsc.1030.0057>
- Cordeau, J.-F., Gendreau, M., Laporte, G., Potvin, J.-Y., & Semet, F. (2002). A guide to vehicle routing heuristics. *Journal of the Operational Research Society*, 53(5), 512–522. <https://doi.org/10.1057/palgrave.jors.2601319>
- Kallehauge, B., Larsen, J., Madsen, O. B. G., & Solomon, M. M. (2005). Vehicle routing problem with time windows. In G. Desaulniers, J. Desrosiers, & M. M. Solomon (Eds.), *Column generation* (pp. 67–98). Springer. https://doi.org/10.1007/0-387-25486-2_3
- Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5), 533–549. [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)
- Braekers, K., Ramaekers, K., & Van Nieuwenhuyse, I. (2016). The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*, 99, 300–313. <https://doi.org/10.1016/j.cie.2015.12.007>
- Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2013). Heuristics for multi-attribute vehicle routing problems: A survey and synthesis. *European Journal of Operational Research*, 231(1), 1–21. <https://doi.org/10.1016/j.ejor.2013.02.053>